GPU-Accelerated Monte Carlo Option Pricing Engine

GPU-Accelerated Monte Carlo Option Pricing Engine

EN605.617 GPU Programming — Course Project Proposal

Author: Daniel Kim **Date:** March 2026

Objective

The goal of this project is to develop a high-performance Monte Carlo simulation engine for pricing financial options using CUDA on an NVIDIA RTX 3060 Ti. The engine will price European and Asian options by simulating millions of random price paths under the Black-Scholes model, serving as a vehicle for applying and benchmarking the GPU programming techniques covered throughout the course (Modules 0–8), including:

- **Thread/block/grid configuration** — Mapping each simulated price path to a CUDA thread, experimenting with 1D block sizes to maximize occupancy on the RTX 3060 Ti (38 SMs, 4864 CUDA cores, compute capability 8.6).
- **Global memory** — Storing per-path simulated price trajectories and payoff results.
- **Shared memory** — Per-block partial sum reductions for computing mean and variance of option payoffs, avoiding costly atomic operations on global memory.
- **Constant memory** — Storing option parameters (strike price, spot price, risk-free rate, volatility, time to expiry) that are uniform across all threads.
- **Registers** — Keeping per-path running price and accumulator variables in registers for maximum throughput.
- **CUDA streams** — Pricing multiple option contracts concurrently on separate streams, overlapping kernel execution with result collection on the host.
- **cuRAND** — Core of the simulation: generating millions of normally distributed random numbers per path using `curand_normal()` for Geometric Brownian Motion steps.
- **Thrust** — Computing aggregate statistics (mean payoff, standard deviation, confidence intervals) via `thrust::reduce` and `thrust::transform_reduce`.
- **Timing and benchmarking** — Using CUDA events to measure kernel execution time across varying path counts, time steps, block sizes, and memory strategies; generating comparative performance data.
- **CLI arguments** — Controlling option parameters (strike, spot, rate, volatility, expiry), number of paths, number of time steps, block size, number of streams, and benchmark mode from the command line.

The deliverable is a single CUDA application that prices options via Monte Carlo simulation, outputs prices with confidence intervals, and includes benchmark data comparing different optimization strategies and a CPU baseline. The application will also render a **“spaghetti plot” visualization** — thousands of simulated price paths drawn as colored lines on a framebuffer, with in-the-money paths (above the strike price) in green and out-of-the-money paths in red. This provides an immediate, intuitive picture of the simulation and makes the GPU’s massive parallelism visually tangible. An animation mode will render frames as the path count increases, showing the price distribution “fill in” over time.

Programming Language / Framework

CUDA (C/C++) — The project will be developed entirely in CUDA C/C++ using the NVIDIA CUDA Toolkit. The target hardware is an NVIDIA GeForce RTX 3060 Ti (8 GB VRAM, compute capability 8.6).

Hypothesis

I expect the following outcomes:

1. **Massive speedup over CPU** — A CPU implementation simulating 10M+ price paths will take seconds to minutes. The GPU implementation should achieve 100–500× speedup by running all paths in parallel, pricing complex options in milliseconds.

2. **Block size sensitivity** — Unlike divergent workloads, Monte Carlo paths have uniform control flow (every thread executes the same loop), so performance should scale smoothly with block size. I hypothesize 256 threads/block will be optimal, balancing occupancy and register pressure.

3. **Constant memory benefit** — Storing option parameters (strike, rate, volatility) in constant memory will yield a measurable speedup over passing them through global memory, since all threads read the same values every time step.

4. **Shared memory reductions** — Using shared memory for per-block partial sum reductions will outperform naive global atomic additions, especially at high path counts where atomic contention becomes significant.

5. **Stream overlap** — Using multiple CUDA streams to price different option contracts concurrently will reduce total wall-clock time by 30–50% compared to sequential pricing, particularly when pricing a portfolio of many options.

6. **Path count convergence** — Increasing the number of simulated paths will improve pricing accuracy (narrower confidence intervals) with diminishing returns, following the expected O(1/√N) convergence rate of Monte Carlo methods.

7. **Visualization as a GPU workload** — Rendering thousands of price paths directly on the GPU (each thread draws its own path's pixels into a shared framebuffer) will be faster than transferring path data to the host for CPU-side rendering, especially at high path counts.

Development Plan (Iterative)

Phase	Milestone	Modules Covered
MVP	European call option pricing with cuRAND paths, CLI args, CPU baseline comparison	0–3 (threads, global/constant memory, CLI), 7 (cuRAND)
Optimization	Shared memory reductions, register tuning, block-size sweep benchmarks	4–5 (shared memory, registers, benchmarking)
Streams	Multi-option portfolio	6 (CUDA streams)

	pricing with concurrent streams	
Statistics	Thrust-based aggregate stats, confidence intervals, convergence analysis	8 (Thrust)
Visualization	Spaghetti path rendering to PPM, animation mode (frames as path count grows)	0–3 (threads, global memory)
Final	Asian option support, performance report, presentation video, code cleanup	—

One-Minute Pitch

I’m building a GPU-accelerated Monte Carlo option pricing engine in CUDA. Each CUDA thread simulates one random price path using cuRAND, and the option payoff is averaged across millions of paths to estimate the fair price. The engine also renders a “spaghetti plot” — thousands of simulated paths drawn as colored lines, green for in-the-money, red for out-of-the-money — so you can literally see the simulation. I’ll use it as a testbed to benchmark all the memory types we’ve covered — constant memory for option parameters, shared memory for per-block reductions, registers for per-path state — plus CUDA streams for pricing multiple options concurrently, and Thrust for computing aggregate statistics and confidence intervals. My hypothesis is that the GPU version will be 100–500× faster than CPU, enabling real-time pricing of option portfolios. The MVP is a European call pricer with path visualization; the stretch goal is Asian option support and animated convergence.